

[Cloud Computing Concepts Hub](#) › Databases

What's the Difference Between Relational and Non-relational Databases?



What's the difference between relational and non-relational databases?



Relational and non-relational databases are two methods of data storage for applications. A relational database (or SQL database) stores data in tabular format with rows and columns. The columns contain data attributes and the rows have data values. You can link the tables in a relational database to gain deeper insights into the interconnection between diverse data points. On the other hand, non-relational databases (or NoSQL databases) use a variety of data models for accessing and managing data. They are optimized specifically for applications that require large data volume, low latency, and flexible data models, which is achieved by relaxing some of the data consistency restrictions of other databases.

[Read about relational databases »](#)

[Read about NoSQL databases »](#)

How do relational databases store data?

Relational databases store data in tables with columns and rows. Each column represents a specific data attribute and each row represents an instance of that data.

You give each table a primary key—an identifier column that uniquely identifies the table. You use the primary key to establish relationships between tables. You use it to relate rows between tables as



foreign key in another table.

Once two tables are connected, you get data from them both with a single query. You write SQL queries to interact with the relational database.

Example of stored data

For example, imagine that a retailer creates a table of all their products. In this table, you could have columns for the product names, descriptions, and price. Another table contains data about customers, their names, and what they have purchased.

The following tables demonstrate this approach.

Product_id (Primary key)	Product_name	Product_cost
--------------------------	--------------	--------------

P1	Product_A	\$100
----	-----------	-------

P2	Product_B	\$50
----	-----------	------

P3	Product_C	\$80
----	-----------	------

Customer_id	Customer_name	Item_purchased (Foreign key)
-------------	---------------	------------------------------

C1	Customer_A	P2
----	------------	----

C2	Customer_B	P1
----	------------	----

C3	Customer_C	P3
----	------------	----



How do non-relational databases store data?

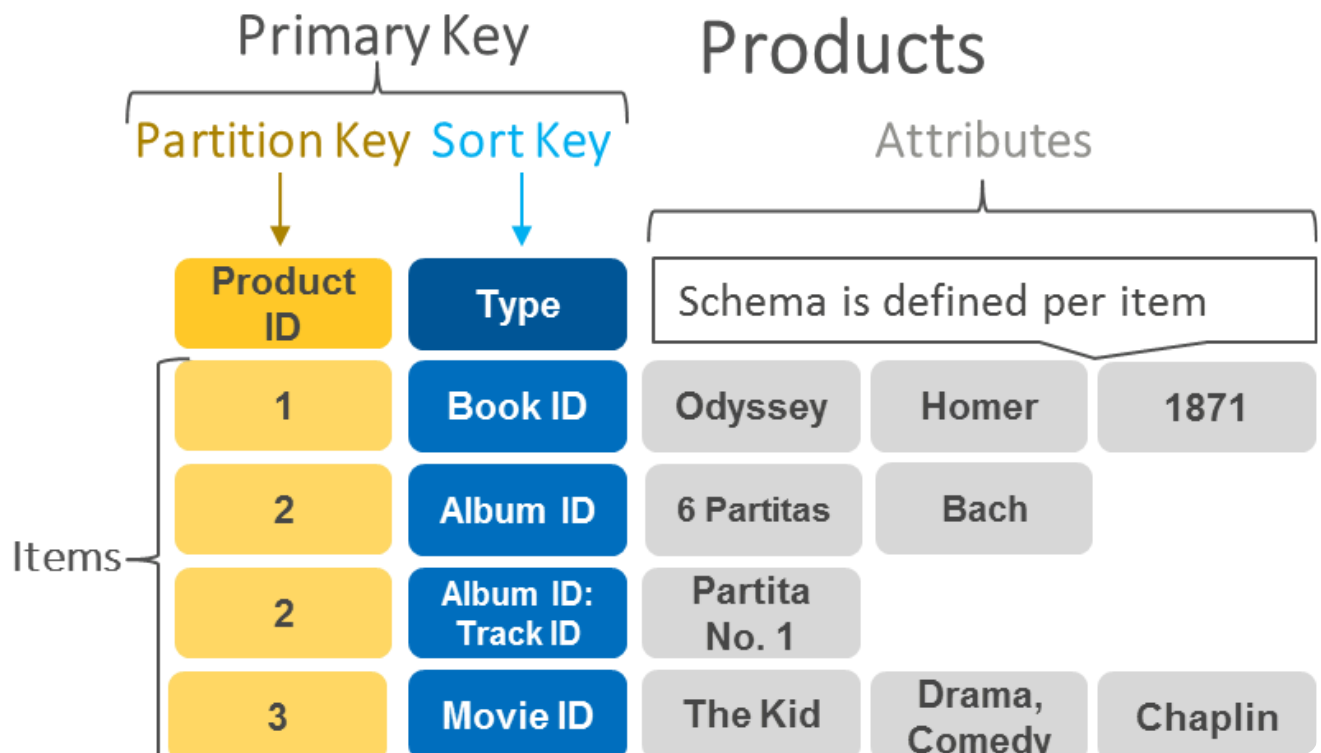
There are several different non-relational database systems due to variations in the way they manage and store schema-less data. Schema-less data is data that's stored without the constraints that relational databases require.

Next, we explain some of the common types of non-relational databases.

Key-value databases

A key-value database stores data as a collection of key-value pairs. In a pair, the key serves as a unique identifier. Both keys and values can be anything from simple objects to complex compound objects.

[Read about key-value databases »](#)



Document databases

Document-oriented databases have the same document model format that developers use in their application code. They store data as JSON objects that are flexible, semi-structured, and hierarchical in nature.



The following example shows what stored data in a document database can look like.

```
{
  company_name: "AnyCompany",
  address: {street: "1212 Main Street", city: "Anytown"},
  phone_number: "1-800-555-0101",
  industry: ["food processing", "appliances"]
  type: "private",
  number_of_employees: 987
}
```

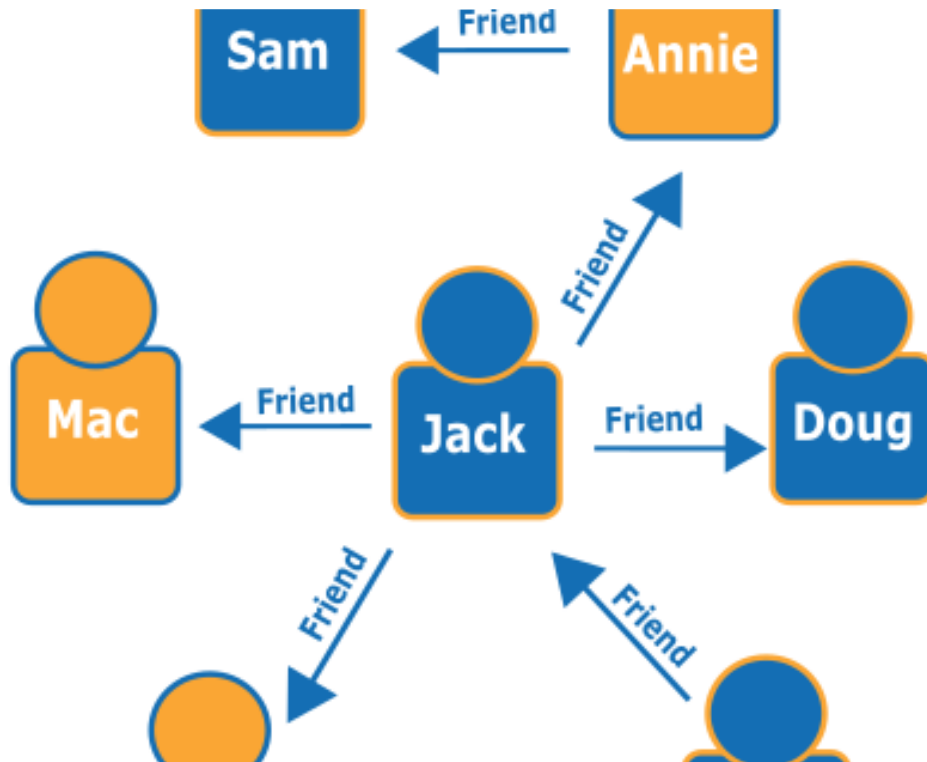
[Read about document databases »](#)

Graph databases

Graph databases are purpose-built to store and navigate relationships. They use nodes to store data entities and edges to store relationships between entities.

An edge always has a start node, end node, type, and direction. It can describe, for example, parent-child relationships, actions, and ownership.

[Read about graph databases »](#)



Key differences: relational vs. non-relational databases

Relational and non-relational databases store and manage data very differently. The following sections discuss specific differences.

Structure

Relational databases store data in tabular form and follow strict rules concerning data variations and table relationships. They allow you to process complex queries on structured data while maintaining data integrity and consistency.

Non-relational databases are more flexible and useful for data with changing requirements. You can use them to store images, videos, documents, and other semi-structured and unstructured content.

Data integrity mechanism

Atomicity, consistency, isolation, and durability (ACID) refer to the database's capability to maintain data integrity despite errors or interruptions in data processing.

A relational database model follows strict ACID properties. This means that a set of consequent operations will always be completed together. If a single operation fails, the entire set of operations fail. This guarantees data accuracy at all times.

In contrast, non-relational databases offer a more flexible model of being basically available, soft-state, and eventually consistent (BASE).

Non-relational databases guarantee availability but not immediate consistency. The database state can change over time and eventually becomes consistent. Some non-relational databases may offer ACID compliance with performance or other trade-offs.

Performance

The performance of relational databases depends on their disk subsystem. To improve database performance, you can use SSDs and optimize the disk by configuring it with a redundant array of independent disks (RAID). To get peak performance, you also have to optimize indexes, table structures, and queries.

In contrast, the performance of NoSQL databases depends on the network latency, hardware cluster size, and the calling application. There are a few ways you can improve the performance of a non-relational database:



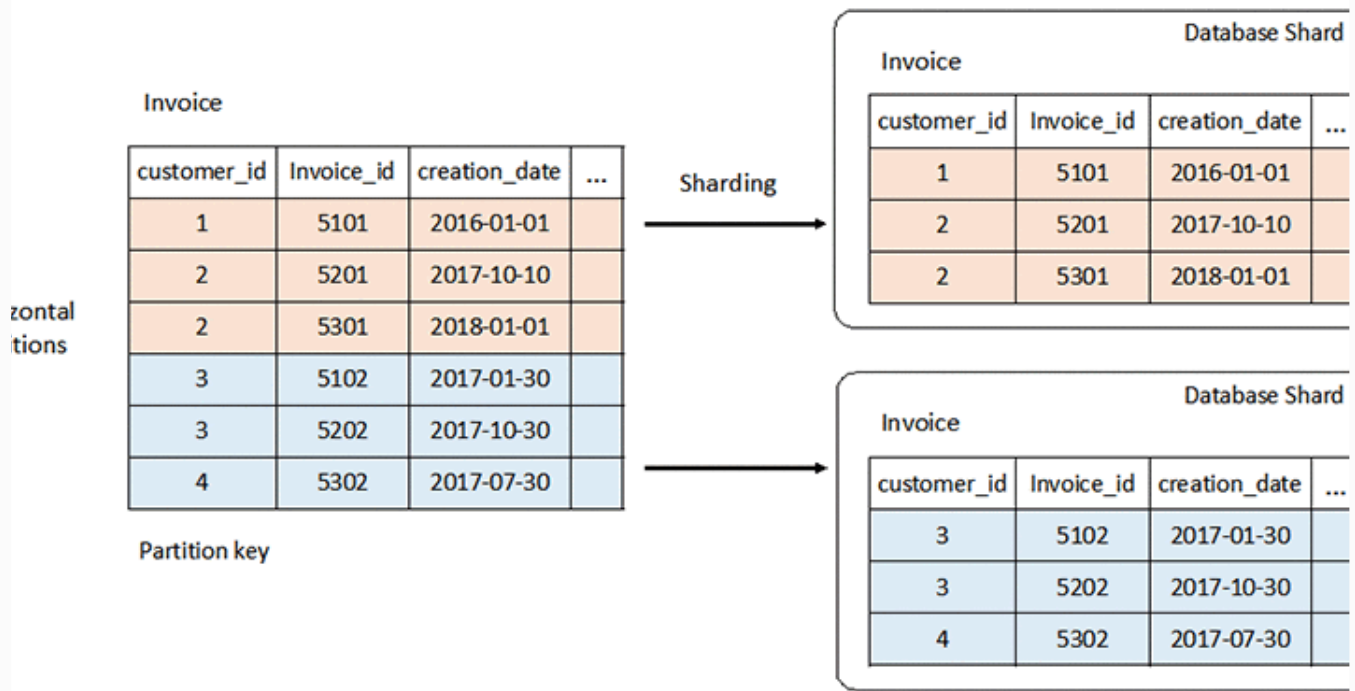
- Increase the cluster size
- Minimize network latency
- Index and cache

NoSQL databases offer higher performance and scalability for specific use cases as compared to a relational database.

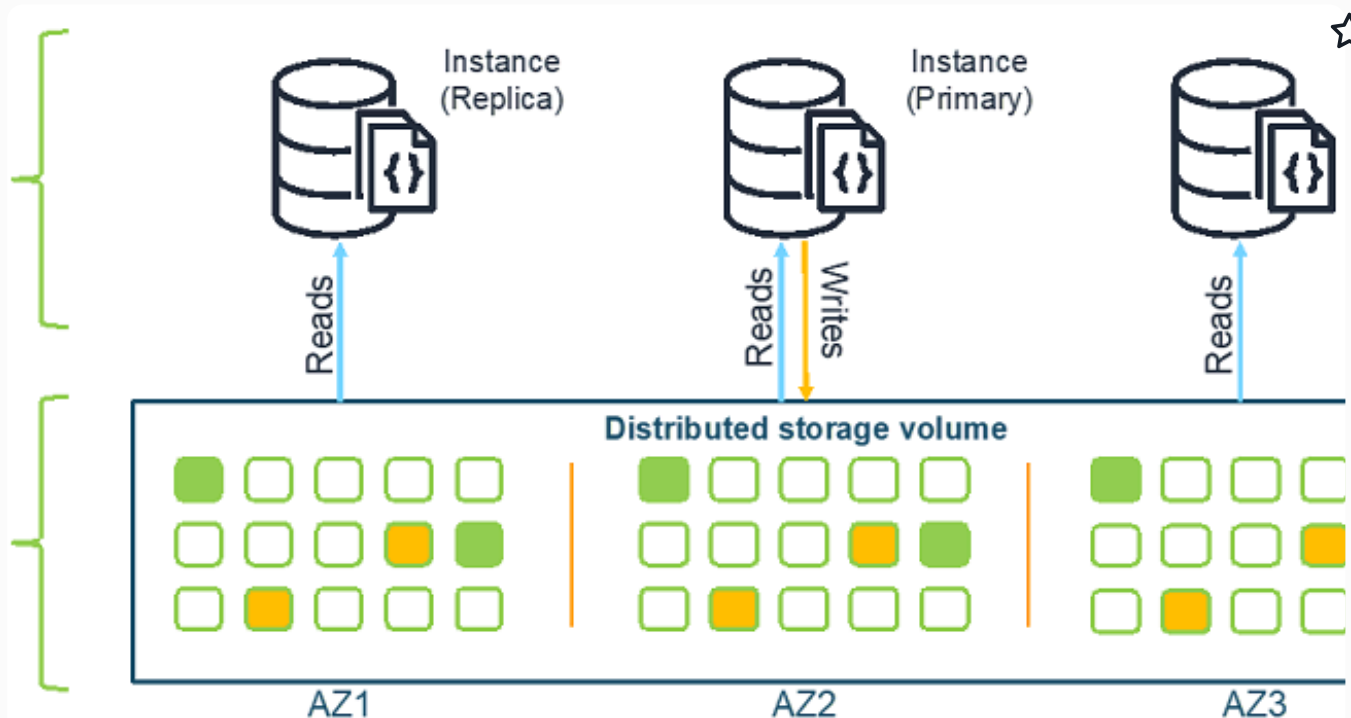
Scale

The rigid schema of a relational database system can present challenges at scale. You typically scale vertically by adding more CPU, or RAM resources, to the server. You can also scale horizontally by duplicating data across servers for read-only workloads. However, horizontal scaling for read-write workloads requires special strategies like partitioning and sharding.

[Read about database sharding »](#)



In contrast, NoSQL databases are highly scalable. You can distribute their workload across many nodes more easily. These databases can process large data volume by partitioning into smaller sets and distributing the sets across multiple nodes.



When to use relational vs. non-relational databases

Relational databases are the best choice if your data is predictable in terms of size, structure, and access frequency. You may also prefer a relational database management system if relationships between entities are important. For example, if you have a large dataset with a complex structure and relationships, you want the relationships to stand out for analytics and ease of use.

In contrast, a non-relational model works better for storing data that is flexible in shape or size, or that may change in the future.

Also, in some cases, data relationships just don't fit well into the tabular primary and foreign keys format. For example, to model the friends and relationships in a social media network, you would need a table with hundreds of rows in a relational database.

In contrast, this can be represented in a single line in a non-relational database. The following example shows data entries for a member with four friends in a non-relational database.

Member_id		
Friend_id		
M1		
M2		
M1	{member name: "member 1"	
M3	member friends: "member 2, member 3, member 4, member 5"	
M1		
M4		
M1		
M5		

Summary of differences: relational vs. non- relational databases

Category	Relational database	Non-relational database
Data model	Tabular.	Key-value, document, or graph.

Data type	Structured.	Structured, semi-structured, and unstructured.
Data integrity	High with full ACID compliance.	Eventual consistency model.
Performance	Improved by adding more resources to the server.	Improved by adding more server nodes.
Scaling	Horizontal scaling requires additional data management strategies.	Horizontal scaling is straightforward.

How can AWS support your relational and non-relational database requirements?

Amazon Web Services (AWS) offers many services for relational and non-relational database requirements.

AWS services for relational databases



[Amazon Relational Database Service \(Amazon RDS\)](#) is a collection of managed services that makes it simple to set up, operate, and scale a relational database in the cloud. Cloud databases offer many benefits like performance, scale, and cost efficiency. You can use relational database engines like these:

- [Amazon RDS for SQL Server](#) to deploy multiple editions of SQL Server (2014, 2016, 2017, and 2019)
- [Amazon RDS for MySQL](#) to support MySQL Community Edition versions 5.7 and 8.0
- [Amazon RDS for MariaDB](#) to support MariaDB Server versions 10.3, 10.4, 10.5, and 10.6

Additionally, [Amazon RDS for Oracle](#) has two different licensing models, which means you don't need separately purchased Oracle licenses if you don't have them.

AWS services for non-relational databases

AWS also has several NoSQL database services to meet all your NoSQL requirements. Here are some examples:

- [Amazon DynamoDB](#) is a key-value database service that provides consistent, single-digit millisecond latency for workloads at any scale.
- [Amazon DocumentDB \(with MondoDB compatibility\)](#) is a popular document-oriented database with powerful and intuitive APIs for flexible and iterative development.
- [Amazon MemoryDB](#) is a durable, in-memory database service. It delivers microsecond read and write latency for ultra-fast performance.

- [Amazon Neptune](#) is a fully managed graph database service to build and run high-performance graph applications.
- [Amazon OpenSearch Service](#) is purpose-built for providing near real-time visualizations and analytics of machine-generated data.

Get started with relational and non-relational databases on AWS by [creating an account](#) today.

Next Steps with AWS

Learn how to get started with Relational Databases on AWS



Learn how to get started with Non-relational Databases



Learn



Resources



Developers



Help





Amazon is an Equal Opportunity Employer: Minority / Women / Disability / Veteran / Gender Identity / Sexual Orientation / Age.



[Back to top](#) ↑

[Privacy](#) [Site terms](#) [Cookie Preferences](#)

© 2025, Amazon Web Services, Inc. or its affiliates. All rights reserved.

